BEOSIN
Web3 Security & Compliance

MolePin

Smart Contract Security Audit

No. 202606191626

Jun 19th, 2026

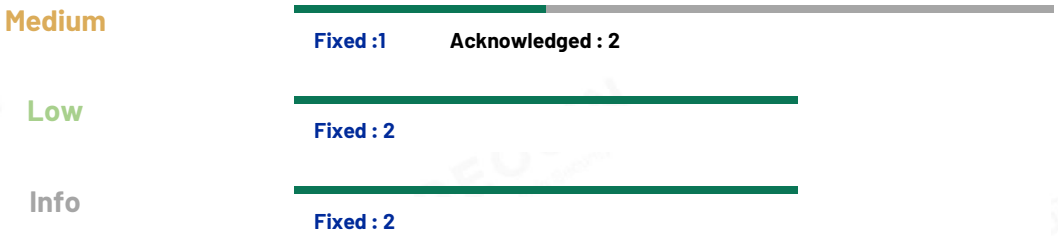
SECURING BLOCKCHAIN ECOSYSTEM

WWW.BEOSIN.COM

Contents

Summary of Audit Results

After auditing, 3 Medium-risk, 2 Low-risk and 2 Info items were identified in the MolePin project. Specific audit details will be presented in the Findings section. Users should pay attention to the following aspects when interacting with this project:



- **Project Description:**

The project under audit is a multi-chain token ecosystem anchored on BSC as the home chain, comprising three core smart contracts: MolePin, MolePinBridgeGateway, and MolePinRemote. The system maintains a fixed total supply through a dual-model cross-chain architecture — lock-and-release on the home chain paired with burn-and-mint on remote chains — built on top of the Chainlink CCIP protocol.

- MolePin is the home-chain token contract deployed on BSC. A fixed supply of 6,942,420,888,888 MOL is minted once at deployment with no subsequent issuance path. The token is deliberately kept pure — no transfer fees, blacklists, or pause mechanisms — ensuring full compatibility with major DEXs and Chainlink CCT liquidity pools. Owner privileges are minimal, limited to a two-step ownership transfer with no ability to affect supply, balances, or transfers.
- MolePinBridgeGateway is the unified bidirectional entry point for all cross-chain transfers, deployed to an identical address across all EVM chains via CREATE2. Chain-specific parameters — local MOL address, CCIP Router, and native-token/USD price feed — are injected once post-deployment via `configure()` and permanently locked. When a user calls `bridge()`, the contract pulls MOL from the caller and dispatches a CCIP message routing tokens through the destination gateway contract rather than directly to the user's EOA, bypassing CCIP's 90,000 gas ceiling on EOA receivers. Upon arrival, `ccipReceive()` decodes the intended recipient from the message payload and forwards the tokens accordingly. A USD-denominated protocol fee (default $1, capped at $5) is charged per transfer, converted to native token in real time via the Chainlink price feed, with any excess native value automatically refunded.
- MolePinRemote is deployed on all non-BSC chains as the remote MOL token implementation, starting with zero supply. Minting and burning are exclusively controlled by a Chainlink CCT BurnMintTokenPool through `MINTER_ROLE` and `BURNER_ROLE`, granted to the pool post-deployment. Remote circulating supply always mirrors the BSC LockRelease pool's locked amount one-to-one, preserving total ecosystem supply across all cross-chain activity. Like the home token, no transfer fees, blacklists, or pause mechanisms are present.

1 Overview

1.1 Project Overview

Project Name	MolePin
Project Language	Solidity
Platform	BSC
File Hash(SHA-256)	e02f930d7a5bf6f5dcddf5e8031bbfe2a56e485985ddabe90b4d952edc95b2ed (Origin) d19631a2d48b824d092e7cc6a7450932dc321d08a92e71a7bfd48bbc3fe4478b (Latest)

1.2 Audit Overview

Audit work duration: Jun 19, 2026 - Jun 19, 2026

Audit team: Beosin Security Team

1.3 Audit Method

The audit methods are as follows:

1. Formal Verification

Formal verification is a technique that uses property-based approaches for testing and verification. Property specifications define a set of rules using Beosin's library of security expert rules. These rules call into the contracts under analysis and make various assertions about their behavior. The rules of the specification play a crucial role in the analysis. If the rule is violated, a concrete test case is provided to demonstrate the violation.

2. Manual Review

Using manual auditing methods, the code is read line by line to identify potential security issues. This ensures that the contract's execution logic aligns with the client's specifications and intentions, thereby safeguarding the accuracy of the contract's business logic.

The manual audit is divided into three groups to cover the entire auditing process:

The Basic Testing Group is primarily responsible for interpreting the project's code and conducting comprehensive functional testing.

The Simulated Attack Group is responsible for analyzing the audited project based on the collected historical audit vulnerability database and security incident attack models. They identify potential attack vectors and collaborate with the Basic Testing Group to conduct simulated attack tests.

The Expert Analysis Group is responsible for analyzing the overall project design, interactions with third parties, and security risks in the on-chain operational environment. They also conduct a review of the entire audit findings.

3. Static Analysis

Static analysis is a function of examining code during compilation or static analysis to detect issues. Beosin-VaaS can detect more than 100 common smart contract vulnerabilities through static analysis, such as reentrancy and block parameter dependency. It allows early and efficient discovery of problems to improve code quality and security.

2 Findings

Index	Risk description	Severity level	Status
MolePin-01	BURNER_ROLE Arbitrary Token Burn	Medium	Acknowledged
MolePin-02	Centralization Risk	Medium	Acknowledged
MolePin-03	Admin Role Not Synced on Ownership Transfer	Medium	Fixed
MolePin-04	trustedRemoteGateway Cannot Be Revoked	Low	Fixed
MolePin-05	No ERC20 Recovery for Stranded Tokens	Low	Fixed
MolePin-06	ccipReceive Missing onlyConfigured	Info	Fixed
MolePin-07	Missing Explicit Access Control	Info	Fixed

Finding Details:

[MolePin-01] BURNER_ROLE Arbitrary Token Burn

Severity Level	Medium
Type	Business Security
Location	MolePinRemote.sol#L97-99
Description	The `burn` function calls `_burn` directly without invoking `_spendAllowance`. Any address holding `BURNER_ROLE` can burn the token balance of an arbitrary account without that account's approval, bypassing standard ERC20 authorization controls. `function burn(address account, uint256 amount) external override onlyRole(BURNER_ROLE) { _burn(account, amount); }`
Recommendation	It is recommended to add `_spendAllowance` inside `burn` to align its behavior with `burnFrom` or delete this `burn` function.
Status	**Acknowledged.** The project team has added comments regarding this function, stating that the permission will only be granted to the designated CCIP pool.

[MolePin-02] Centralization Risk

Severity Level	Medium
Type	Business Security
Location	MolePinBridgeGateway.sol / MolePinRemote.sol
Description	All privileged functions — including `configure()`, `setFeeUsd()`, `setTreasury()`, `setDestChain()`, `setTrustedRemoteGateway()`, `sweepNative()`, and role management — are controlled by a single owner address with no time delay or multi-party approval requirement.
Recommendation	It is recommended to transfer contract ownership to a multi-signature wallet.
Status	**Acknowledged.**

[MolePin-03] Admin Role Not Synced on Ownership Transfer

Severity Level	**Medium**
Type	Business Security
Location	MolePinRemote.sol#L59
Description	`DEFAULT_ADMIN_ROLE` is granted only to `initialOwner` in the constructor and is not automatically migrated during a Ownable2Step ownership transfer. After the new owner accepts ownership, they lack `DEFAULT_ADMIN_ROLE` and cannot call `grantPoolRoles` or `grantMintAndBurnRoles`, causing role management authority to diverge from contract ownership.
Recommendation	It is recommended to override `_transferOwnership` to atomically revoke `DEFAULT_ADMIN_ROLE` from the outgoing owner and grant it to the incoming owner upon ownership acceptance.
Status	**Fixed.** The `_transferOwnership` function has been rewritten to synchronously transfer roles.

```
    constructor(address initialOwner)
        ERC20("MolePin", "MOL")
        ERC20Permit("MolePin")
        Ownable(initialOwner)
    {
        _grantRole(DEFAULT_ADMIN_ROLE, initialOwner);
        // No mint here. Remote starts at 0 -> pool mints only what is
bridged in.
    }
```

```
    function _transferOwnership(address newOwner) internal override {
        address previousOwner = owner();
        super._transferOwnership(newOwner);
        if (newOwner != previousOwner) {
            if (previousOwner != address(0)) {
                _revokeRole(DEFAULT_ADMIN_ROLE, previousOwner);
            }
            if (newOwner != address(0)) {
                _grantRole(DEFAULT_ADMIN_ROLE, newOwner);
            }
        }
    }
```

[MolePin-04] trustedRemoteGateway Cannot Be Revoked

Severity Level	Low
Type	Business Security
Location	MolePinBridgeGateway.sol#L340-345

Description	`setTrustedRemoteGateway` enforces `gateway != address(0)`, making it impossible to fully revoke trust for a remote chain by zeroing the entry. Since `ccipReceive` only checks `trustedRemoteGateway` and does not check `allowedDestChains`, disabling a chain via `setDestChain(selector, false)` stops outbound bridging but inbound messages from that chain continue to be accepted.
	`function setTrustedRemoteGateway(uint64 selector, address gateway) external onlyOwner {`
	`require(gateway != address(0), "zero addr");`
	`trustedRemoteGateway[selector] = gateway;`
	`emit TrustedRemoteUpdated(selector, gateway);`
	`}`

Recommendation	It is recommended to remove the require restriction to allow zero-address assignment as a revocation mechanism.

Status	**Fixed.**
	`function setTrustedRemoteGateway(uint64 selector, address gateway) external onlyOwner {`
	`trustedRemoteGateway[selector] = gateway;`
	`emit TrustedRemoteUpdated(selector, gateway);`
	`}`

[MolePin-05] No ERC20 Recovery for Stranded Tokens

Severity Level	Low
Type	Business Security
Location	MolePinBridgeGateway.sol
Description	The gateway contract holds MOL transiently during `bridge()` execution and serves as the designated CCIP receiver for all inbound cross-chain messages. If a CCIP message permanently fails on the destination side and Chainlink's recovery mechanism returns the tokens to the gateway, or if MOL is sent to the gateway directly by mistake, there is no function to extract stranded ERC20 tokens. `sweepNative` covers native token recovery but no equivalent exists for ERC20 assets.
Recommendation	It is recommended to add a permissioned rescueERC20 function.
Status	**Fixed.**

```
    function rescueERC20(address token, address to, uint256 amount)
external onlyOwner {
        require(token != address(0) && to != address(0), "zero addr");
        IERC20(token).safeTransfer(to, amount);
        emit ERC20Rescued(token, to, amount);
    }
```

[MolePin-06] ccipReceive Missing onlyConfigured

Severity Level	Info
Type	Coding Conventions
Location	MolePinBridgeGateway.sol#L291

Description	`ccipReceive` does not apply the `onlyConfigured` modifier. When the contract is unconfigured, ROUTER == address(0), so any call reverts with "only router" rather than "not configured". While no security vulnerability exists, the misleading error message obscures the true cause during debugging and incident investigation.
	`function ccipReceive(Client.Any2EVMMessage calldata message) external override nonReentrant {`
	`require(msg.sender == address(ROUTER), "only router");`

Recommendation	It is recommended to apply the `onlyConfigured` modifier to `ccipReceive` so that failures in the unconfigured state produce a clear and accurate error message.

Status	**Fixed.**
	`function ccipReceive(Client.Any2EVMMessage calldata message) external override nonReentrant onlyConfigured {`
	`require(msg.sender == address(ROUTER), "only router");`

[MolePin-07] Missing Explicit Access Control

Severity Level	Info
Type	Coding Conventions
Location	MolePinRemote.sol#L76-79

Description	`grantMintAndBurnRoles` carries no explicit access control modifier such as `onlyRole(DEFAULT_ADMIN_ROLE)`. Its restriction relies entirely on the internal permission check inside OZ's `grantRole`. This makes the access intent opaque at the function signature level, increases the risk of misuse in future modifications.
	`function grantMintAndBurnRoles(address burnAndMinter) external {`
	`grantRole(MINTER_ROLE, burnAndMinter);`
	`grantRole(BURNER_ROLE, burnAndMinter);`
	`}`

Recommendation	It is recommended to add an explicit `onlyRole(DEFAULT_ADMIN_ROLE)` modifier to make the access restriction self-evident at the function signature level.

Status	**Fixed.**
	`function grantMintAndBurnRoles(address burnAndMinter) external onlyRole(DEFAULT_ADMIN_ROLE) {`
	`_grantRole(MINTER_ROLE, burnAndMinter);`
	`_grantRole(BURNER_ROLE, burnAndMinter);`
	`}`

3 Appendix

3.1 Vulnerability Assessment Metrics and Status in Smart Contracts

3.1.1 Metrics

In order to objectively assess the severity level of vulnerabilities in blockchain systems, this report provides detailed assessment metrics for security vulnerabilities in smart contracts with reference to CVSS 3.1(Common Vulnerability Scoring System Ver 3.1).

According to the severity level of vulnerability, the vulnerabilities are classified into four levels: "critical", "high", "medium" and "low". It mainly relies on the degree of impact and likelihood of exploitation of the vulnerability, supplemented by other comprehensive factors to determine of the severity level.

Likelihood \ Impact	Severe	High	Medium	Low
Probable	Critical	High	Medium	Low
Possible	High	Medium	Medium	Low
Unlikely	Medium	Medium	Low	Info
Rare	Low	Low	Info	Info

3.1.2 Degree of impact

- **Severe**

Severe impact generally refers to the vulnerability can have a serious impact on the confidentiality, integrity, availability of smart contracts or their economic model, which can cause substantial economic losses to the contract business system, large-scale data disruption, loss of authority management, failure of key functions, loss of credibility, or indirectly affect the operation of other smart contracts associated with it and cause substantial losses, as well as other severe and mostly irreversible harm.

- **High**

High impact generally refers to the vulnerability can have a relatively serious impact on the confidentiality, integrity, availability of the smart contract or its economic model, which can cause a greater economic loss, local functional unavailability, loss of credibility and other impact to the contract business system.

- **Medium**

Medium impact generally refers to the vulnerability can have a relatively minor impact on the confidentiality, integrity, availability of the smart contract or its economic model, which can cause a small amount of economic loss to the contract business system, individual business unavailability and other impact.

- **Low**

Low impact generally refers to the vulnerability can have a minor impact on the smart contract, which can pose certain security threat to the contract business system and needs to be improved.

3.1.3 Likelihood of Exploitation

- **Probable**

Probable likelihood generally means that the cost required to exploit the vulnerability is low, with no special exploitation threshold, and the vulnerability can be triggered consistently.

- **Possible**

Possible likelihood generally means that exploiting such vulnerability requires a certain cost, or there are certain conditions for exploitation, and the vulnerability is not easily and consistently triggered.

- **Unlikely**

Unlikely likelihood generally means that the vulnerability requires a high cost, or the exploitation conditions are very demanding and the vulnerability is highly difficult to trigger.

- **Rare**

Rare likelihood generally means that the vulnerability requires an extremely high cost or the conditions for exploitation are extremely difficult to achieve.

3.1.4 Fix Results Status

Status	Description
Fixed	The project party fully fixes a vulnerability.
Partially Fixed	The project party did not fully fix the issue, but only mitigated the issue.
Acknowledged	The project party confirms and chooses to ignore the issue.

3.2 Audit Categories

No.	Categories	Subitems
1	Coding Conventions	Compiler Version Security
		Deprecated Items
		Redundant Code
		require/assert Usage
		Gas Consumption
2	General Vulnerability	Integer Overflow/Underflow
		Reentrancy
		Pseudo-random Number Generator (PRNG)
		Transaction-Ordering Dependence
		DoS (Denial of Service)
		Function Call Permissions
		call/delegatecall Security
		Returned Value Security
		tx.origin Usage
		Replay Attack
		Overriding Variables
		Third-party Protocol Interface Consistency
3	Business Security	Business Logics
		Business Implementations
		Manipulable Token Price
		Centralized Asset Control
		Asset Tradability
		Arbitrage Attack

Beosin classified the security issues of smart contracts into three categories: Coding Conventions, General Vulnerability, Business Security. Their specific definitions are as follows:

- **Coding Conventions**

Audit whether smart contracts follow recommended language security coding practices. For example, smart contracts developed in Solidity language should fix the compiler version and do not use deprecated keywords.

- **General Vulnerability**

General Vulnerability include some common vulnerabilities that may appear in smart contract projects. These vulnerabilities are mainly related to the characteristics of the smart contract itself, such as integer overflow/underflow and denial of service attacks.

- **Business Security**

Business security is mainly related to some issues related to the business realized by each project, and has a relatively strong pertinence. For example, whether the lock-up plan in the code match the white paper, or the flash loan attack caused by the incorrect setting of the price acquisition oracle.

*Note that the project may suffer stake losses due to the integrated third-party protocol. This is not something Beosin can control. Business security requires the participation of the project party. The project party and users need to stay vigilant at all times.

3.3 Disclaimer

The Audit Report issued by Beosin is related to the services agreed in the relevant service agreement. The Project Party or the Served Party (hereinafter referred to as the "Served Party") can only be used within the conditions and scope agreed in the service agreement. Other third parties shall not transmit, disclose, quote, rely on or tamper with the Audit Report issued for any purpose.

The Audit Report issued by Beosin is made solely for the code, and any description, expression or wording contained therein shall not be interpreted as affirmation or confirmation of the project, nor shall any warranty or guarantee be given as to the absolute flawlessness of the code analyzed, the code team, the business model or legal compliance.

The Audit Report issued by Beosin is only based on the code provided by the Served Party and the technology currently available to Beosin. However, due to the technical limitations of any organization, and in the event that the code provided by the Served Party is missing information, tampered with, deleted, hidden or subsequently altered, the audit report may still fail to fully enumerate all the risks.

The Audit Report issued by Beosin in no way provides investment advice on any project, nor should it be utilized as investment suggestions of any type. This report represents an extensive evaluation process designed to help our customers improve code quality while mitigating the high risks in blockchain.

3.4 About Beosin

Beosin is a leading blockchain security and compliance technology company established in 2018. Being focused on blockchain ecosystem security and compliance, it has developed a product matrix including Beosin KYT, Beosin Trace, and Stablecoin Monitor, which have obtained international certifications such as ISO 27001 and SOC 2. Beosin's core products have been applied for over 70 intellectual property rights, and the company has participated in the development of multiple international standards related to blockchain security. It was among the first batch of enterprises selected for the Cyberport Incubation Programme. Its business covers professional code security audit services for blockchain ecosystems, anti-money laundering compliance technology services for exchanges, financial institutions, and payment institutions, and virtual asset tracing and investigation services for law enforcement and regulatory authorities.

As one of the earliest companies to apply formal verification to blockchain security, Beosin offers professional blockchain and smart contract security audit services. Beosin has audited over 4,500 smart contracts and blockchain projects and has become the official security partner for several renowned blockchains, including BNB Chain, TON, Soneium, Manta Network, Sonic SVM, and SOON Network.

BEOSIN
Web3 Security & Compliance

Official Website
https://www.beosin.com

Telegram
https://t.me/beosin

X
https://x.com/Beosin_com

Email
service@beosin.com

LinkedIn
https://www.linkedin.com/company/beosin